Question 4: Docker Network - Demonstrate communication between two containers through a Docker network, using container names and container IP addresses.

A user-defined bridge network was created with two containers. They communicated by container name (embedded DNS) and by IP address. The default bridge network was used as a contrast: it has no embedded DNS.

Network and containers

```
$ docker network create lab2net
$ docker run -d --name net-a --network lab2net ubuntu:24.04 sleep infinity
$ docker run -d --name net-b --network lab2net ubuntu:24.04 sleep infinity

$ docker inspect net-a --format "{{.NetworkSettings.Networks.lab2net.IPAddress}}"
net-a IP: 172.18.0.2
$ docker inspect net-b --format "{{.NetworkSettings.Networks.lab2net.IPAddress}}"
net-b IP: 172.18.0.3
```

Communication by container name (embedded DNS)

```
$ docker exec net-a ping -c 2 net-b
PING net-b (172.18.0.3) 56(84) bytes of data.
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=1 ttl=64 time=9.50 ms
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=2 ttl=64 time=0.090 ms
2 packets transmitted, 2 received, 0% packet loss
```

Communication by container IP address

```
$ docker exec net-a ping -c 2 172.18.0.3
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.087 ms
64 bytes from 172.18.0.3: icmp_seq=2 ttl=64 time=0.116 ms
2 packets transmitted, 2 received, 0% packet loss
```

Contrast: default bridge network has no embedded DNS

```
$ docker run -d --name net-c ubuntu:24.04 sleep infinity
$ docker exec net-c ping -c 1 net-a
ping: net-a: Name or service not known
--> user-defined networks are required for name resolution
```

```
abhinav@LAPTOP-BSV7PCJK: docs/commands  —  terminal
### Two containers talking over a user-defined bridge network
abhinav@LAPTOP-BSV7PCJK:~$ docker network create lab2net
abhinav@LAPTOP-BSV7PCJK:~$ docker network ls | grep lab2net
0f6f494d736a   lab2net    bridge    local
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name net-a --network lab2net ubuntu:24.04 sleep infinity
78803bf67a504aeeb6d3423b4cebe4d317ae2c106fa97719ccbc546bbd3b5232
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name net-b --network lab2net ubuntu:24.04 sleep infinity
245105f4700b21dff87ecd8fbbb3c42c150565d8c7e7c22257706ffcf0607049
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-a apt-get install -y -qq iputils-ping   (minimal image: add pi
ping installed in both containers
abhinav@LAPTOP-BSV7PCJK:~$ docker inspect net-a --format "net-a IP: {{.NetworkSettings.Networks.lab2net.I
net-a IP: 172.18.0.2
abhinav@LAPTOP-BSV7PCJK:~$ docker inspect net-b --format "net-b IP: {{.NetworkSettings.Networks.lab2net.I
net-b IP: 172.18.0.3
### Communication using the container NAME (embedded DNS)
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-a ping -c 2 net-b
PING net-b (172.18.0.3) 56(84) bytes of data.
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=1 ttl=64 time=9.50 ms
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=2 ttl=64 time=0.090 ms
--- net-b ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 0.090/4.792/9.495/4.702 ms
### Communication using the container IP ADDRESS
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-a ping -c 2 172.18.0.3
PING 172.18.0.3 (172.18.0.3) 56(84) bytes of data.
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.087 ms
64 bytes from 172.18.0.3: icmp_seq=2 ttl=64 time=0.116 ms
--- 172.18.0.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1014ms
rtt min/avg/max/mdev = 0.087/0.101/0.116/0.014 ms
### Containers on the DEFAULT bridge network cannot resolve each other by name
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name net-c ubuntu:24.04 sleep infinity
55e7620ea0cf21711585a65fd999571bdc9175eebdac7962bb405c4dd78faba7
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-c ping -c 1 net-a
ping: net-a: Name or service not known
--> default bridge has no embedded DNS: user-defined networks are required for name resolution
```

Fig: Full network demo: name resolution, IP ping and the default-bridge contrast

Observations

- User-defined bridge networks provide automatic DNS between containers; the default bridge does not.
- IP-based communication works on any network but IPs change when containers are recreated - names are stable.
- The minimal ubuntu:24.04 image ships without ping; iputils-ping was installed inside the demo containers.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Docker, Docker Compose, Minikube and kubectl. The evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-2 under the docs/ directory.